

Operating Systems

Top 50 Interview Questions & Deep-Dive Solutions

A Comprehensive Technical Preparation Guide
Built for Software Engineers & Systems Architects

1. Core Concepts & Architecture

Q1: What is an Operating System, and what are its primary functions?

An **Operating System (OS)** is a core system software that acts as an intermediary interface between the computer hardware and the user applications. Its primary functions include:

- **Resource Management:** Allocating CPU time, memory space, and peripheral devices dynamically to various running processes.
- **Process Management:** Scheduling, creating, synchronizing, and terminating processes or threads.
- **Memory Management:** Tracking active memory frames, loading data into physical RAM, and handling mapping from virtual to physical spaces.
- **File System Control:** Organizing unstructured raw bits on physical storage disks into structured directories and files.
- **Security and Isolation:** Protecting concurrent applications from interfering with one another or accessing unauthorized hardware instructions.

Q2: Explain the difference between Kernel Mode and User Mode. Why is this separation necessary?

The CPU switches between two primary operational execution modes to guarantee system stability and crash-resistance:

- **User Mode:** The mode in which application software executes. Code has limited access to hardware instructions and cannot directly modify system memory or access I/O devices. If a program crashes, it only kills itself, leaving the system safe.
- **Kernel Mode (Supervisor/Privileged Mode):** The execution state of the core OS components. Code running in this mode has unrestricted, full access to the underlying hardware, physical memory registers, and privileged CPU instructions.

Why it is necessary: Without this architectural boundary, any normal user program could overwrite kernel structures, read confidential system RAM, or crash the computer hardware via erratic instruction execution.

Q3: What is a System Call? Walk through what happens when a program executes a system call.

A **System Call** is the programmatic way an application requests a privileged service from the Operating System kernel (e.g., reading a file, sending data over a network socket, creating a child process).

Execution Walkthrough:

1. The user application sets up system call parameters (usually in CPU registers) and executes a software interrupt or a dedicated assembly trap instruction (like `syscall` or `int 0x80`).
2. The CPU transitions from **User Mode** to **Kernel Mode** via a hardware switch mechanism.

3. The hardware looks up the appropriate handler in the **Interrupt Vector Table / System Call Table** using the provided ID.
4. The kernel executes the privileged request code safely.
5. Once complete, the kernel places return values in registers and executes a return-from-trap instruction, switching the execution state back to **User Mode**.

Q4: Compare a Monolithic Kernel vs. a Microkernel. What are the pros and cons of each?

- **Monolithic Kernel (e.g., Linux, Windows):** The entire operating system logic (VFS, memory manager, process scheduler, device drivers) resides inside a single massive binary running entirely in a unified Kernel space.

Pros: Extremely fast due to minimal context switching; functions call each other inline.

Cons: High vulnerability; if a single third-party driver crashes, the whole kernel panics.

- **Microkernel (e.g., QNX, L4):** Strips kernel features down to absolute bare-minimum primitives (IPC, low-level scheduling, basic memory management). File systems and drivers run as unprivileged servers in User space.

Pros: High modularity, extensibility, and extreme crash resilience (if a driver crashes, you just restart its user process).

Cons: Performance penalties due to heavy Inter-Process Communication (IPC) context-switching overhead.

Q5: What is Dual-Mode operation in an OS, and how does the hardware support it?

Dual-Mode operation divides hardware capabilities into at least two distinct operational privileges: User Mode and Monitor/Kernel Mode.

Hardware Support: It is enforced via a structural **Mode Bit** inside the CPU's processor status word (PSW) register. The bit defaults to 1 for User Mode and 0 for Kernel Mode. If an attempt is made to invoke an architectural instruction marked "privileged" (such as disabling all physical hardware interrupts) while the mode bit is 1, the hardware automatically halts execution and triggers an illegal operation trap back into the OS.

Q6: Explain the concept of Virtualization in the context of operating systems.

Virtualization is an architectural abstraction mechanism where physical hardware assets (CPUs, storage, memory controllers) are split into logical, independent entities via a piece of software called a **Hypervisor** or **Virtual Machine Monitor (VMM)**.

Type-1 Hypervisors run straight on the bare hardware fabric, while Type-2 Hypervisors run on top of a host OS. Virtualization isolates Guest OS instances inside independent, virtualized sandboxes by trapping and emulating their privileged instructions safely.

Q7: What is a bootstrapping program (Bootloader), and where is it stored?

The **Bootloader** is the first piece of software executed by the CPU upon powering up a machine. It initializes basic motherboards, identifies available non-volatile system drives, and loads the kernel image into RAM.

before handing over CPU control.

It is stored inside non-volatile firmware chips on the motherboard, conventionally using legacy **BIOS** or modern **UEFI** (Unified Extensible Firmware Interface) non-volatile ROM/Flash chips.

Q8: What is RTOS (Real-Time Operating System)? Distinguish between "Hard" and "Soft" real-time systems.

An **RTOS** is an operating system explicitly optimized for computing systems where timing constraints and deterministic execution behavior are absolute design priorities rather than raw average throughput.

- **Hard RTOS:** Missing a processing deadline guarantees critical system failure or physical danger (e.g., automotive ABS brakes, pacemaker nodes, missile guidance loops). No late data is accepted.
- **Soft RTOS:** Deadlines are strictly pursued, but occasional violations merely degrade QoS rather than causing total failure (e.g., video streaming playback, voice communication).

Q9: What are Interrupts and Exceptions? How does the OS handle an interrupt?

- **Interrupt:** An asynchronous signal triggered from an external hardware device (like a keyboard key press or disk controller finish) indicating it needs service.
- **Exception:** A synchronous event generated inside the CPU itself by the executing code when a serious error or validation state occurs (e.g., division by zero, page fault).

Handling Process: The CPU finishes the current hardware-level instruction, saves the current program counter (PC) and processor state on the kernel stack, looks up the specific handler function using the interrupt number inside the **Interrupt Vector Table**, jumps to and executes the Interrupt Service Routine (ISR), and then resumes the saved program state.

Q10: What is DMA (Direct Memory Access), and why is it vital for high-performance I/O?

DMA is a specialized hardware capability that allows I/O subsystems (like network cards or NVMe controllers) to read and write blocks of data directly from/to main system RAM without constantly passing data through the CPU.

Why it is vital: Without DMA, the CPU would have to execute a polling loop or handle an individual interrupt for every single byte or word transferred, pinning CPU utilization to 100% just for standard disk operations. DMA frees up the processor to handle application code while hardware data streams in parallel.

2. Process Management & CPU Scheduling

Q11: What is a Process, and how does it differ from a Program?

A **Program** is an inactive, passive block of compiled machine code stored as an executable file on a static physical storage drive (e.g., an .exe or binary asset on disk).

A **Process** is an active, dynamic instance of that program currently loaded in RAM and executing. It possesses dynamic execution context, including allocated memory blocks, a dedicated program counter, CPU registers, file descriptors, and active threads.

Q12: Explain the different states of a process (New, Ready, Running, Waiting, Terminated).

- **New:** The process is currently being allocated memory structures and instantiated.
- **Ready:** The process is loaded in memory and waiting inside a queue to be assigned to a CPU core by the scheduler.
- **Running:** The instructions are actively being read and executed by the physical CPU core.
- **Waiting (Blocked):** The process is suspended waiting for an event or I/O operation completion.
- **Terminated:** The process has completed execution, and its memory assets are being reclaimed.

Q13: What is a Process Control Block (PCB)? What specific information does it hold?

A **PCB** is a vital data structure managed by the kernel for every active process. It tracks everything needed to manage and resume a process:

- **Process ID (PID)** and parent-child hierarchy linkages.
- **Process State** (Ready, Running, Blocked, etc.).
- **CPU Registers:** Saved state of accumulation registers, stack pointers, and indices when the process is suspended.
- **Program Counter (PC):** Location of the next instruction to execute.
- **Memory Limits:** Page tables or base/limit register values.
- **I/O Status:** List of opened file handles, sockets, and pending hardware requests.

Q14: What is Context Switching? Why is it considered an overhead?

Context Switching is the mechanism where the kernel saves the execution state (context) of a currently running process or thread, and loads the previously saved state of another process to resume execution on a CPU core.

Why it is overhead: No useful user computations are performed during a context switch. It involves heavy memory access overhead to update the PCB, swap CPU registers, change memory map roots, and invalidates modern hardware cache paths (such as the L1/L2 caches and TLB entries).

Q15: What is a Thread? Explain the difference between a process and a thread, and when you would use one over the other.

A **Thread** is a basic unit of CPU utilization, often called a lightweight process. Multiple threads belong to a single parent process.

Differences:

- Processes do not share memory segments by default; threads share the exact same Text (Code), Data, and Heap sections, but maintain unique Stacks, Registers, and Program Counters.
- Process creation and switching are slow; thread creation and switching are fast.

When to use: Use **threads** for highly cooperative tasks requiring rapid data sharing and fast concurrency (e.g., parallel matrix multiplication). Use **processes** when strict isolation, safety, and security are paramount, preventing a failure in one worker from compromising another (e.g., separate tabs in modern web browsers).

Q16: Differentiate between User-level threads and Kernel-level threads.

- **User-level threads:** Managed entirely by a user-space threading library without any direct awareness from the kernel.

Trade-off: Fast creation and scheduling because no system calls are made. However, if one user thread blocks on I/O, the entire kernel process blocks, freezing all other user threads in that group.

- **Kernel-level threads:** Maintained, scheduled, and recognized directly by the OS kernel.

Trade-off: Slower to create and swap due to system call transitions. However, if one thread blocks, the kernel scheduler can independently swap in another thread from the same process on a different core.

Q17: What is the difference between a Short-term, Medium-term, and Long-term Scheduler?

- **Long-term Scheduler (Job Scheduler):** Controls the ****degree of multiprogramming**** by choosing which newly initiated programs are brought from disk storage into the main memory "Ready" queue. Runs infrequently.
- **Short-term Scheduler (CPU Scheduler):** Picks which process from the active in-memory "Ready" queue should be allocated execution time on the CPU next. Runs incredibly fast (millisecond scales).
- **Medium-term Scheduler (Swapper):** Dynamically manages memory pressure by temporarily removing active processes from RAM and writing them out to swap space on disk ("swapping out"), then restoring them later ("swapping in").

Q18: Explain the following CPU scheduling algorithms: FCFS, SJF, Round Robin, and Priority Scheduling.

- **FCFS (First-Come, First-Served):** Non-preemptive scheduling based simply on arrival timestamps. Simple to build but prone to performance bottlenecks.
- **SJF (Shortest Job First):** Selects the process with the smallest subsequent CPU burst. Provably optimal for minimizing average waiting time, but hard to implement because predicting future CPU burst durations is difficult.
- **Round Robin (RR):** A preemptive algorithm designed for time-sharing. Every process is granted a fixed execution slice of time (Time Quantum q). If it isn't done, it is moved to the back of the queue.
- **Priority Scheduling:** Assigns an integer priority value to each process. The CPU is allocated to the process with the highest priority rank. Can be preemptive or non-preemptive.

Q19: What is the Starvation problem in CPU scheduling, and how does Aging solve it?

Starvation occurs when low-priority or long-duration processes are indefinitely delayed from executing because high-priority or short-duration processes are constantly arriving and taking the CPU.

Solution (Aging): Aging is a progressive technique that gradually increments the internal priority metric of a process the longer it waits in the Ready queue. Eventually, even the lowest-priority task will accumulate a high enough priority score to claim the CPU.

Q20: What is Convoy Effect in FCFS scheduling?

The **Convoy Effect** occurs in FCFS scheduling when a single massive, CPU-bound process blocks the CPU for a long duration, forcing a long line of short, I/O-bound processes to wait indefinitely in the Ready queue. This severely degrades overall device throughput and average response times.

Q21: What is a Preemptive vs. Non-preemptive scheduling algorithm?

- **Non-preemptive:** Once a process is allocated the CPU, it holds it until it voluntarily surrenders control (either by terminating or transitioning into a Waiting state for I/O).
- **Preemptive:** The OS kernel can interrupt a running process mid-execution and reallocate the CPU core to another process (e.g., when a higher priority task arrives or its time slice expires).

Q22: What is a Multilevel Queue and Multilevel Feedback Queue scheduling?

- **Multilevel Queue:** Partitions the Ready queue into distinct, fixed classes (e.g., Foreground interactive processes vs. Background batch tasks). Each queue has its own strict scheduling logic, and tasks cannot move between queues.
- **Multilevel Feedback Queue (MLFQ):** Allows processes to dynamically migrate between multiple queues based on their real-time behavior. For instance, if a process utilizes the CPU for too long, it is downgraded to a lower-priority queue. If it yields frequently for I/O, it is upgraded to a high-priority interactive queue, balancing throughput and responsiveness.

3. Process Synchronization & Inter-Process Communication

Q23: What is Inter-Process Communication (IPC)? Explain the two primary models: Shared Memory and Message Passing.

IPC refers to the set of mechanisms provided by an OS that allow distinct processes to safely share data and coordinate activities.

- **Shared Memory Model:** The OS maps a common physical memory segment into the address spaces of multiple processes. They can read and write directly to it at hardware speeds.

Pros/Cons: Fastest mechanism, but requires user-level synchronization (like mutexes) to avoid race conditions.

- **Message Passing Model:** Processes communicate by sending and receiving explicit packets via kernel-managed system calls (e.g., pipes, sockets, message queues).

Pros/Cons: Easier to implement and inherently safe across networks, but carries higher system call and copying overhead.

Q24: What is the Critical Section Problem, and what three requirements must any solution satisfy?

The **Critical Section** is a segment of code where shared resources (global variables, files) are mutated. Concurrent access can lead to unpredictable data corruption.

Any valid software or hardware solution must satisfy three strict requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other processes can execute in their critical sections for that same shared resource.
2. **Progress:** If no process is in its critical section and some processes want to enter, only those processes not executing in their remainder sections can participate in deciding which process enters next, and this selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There must be a limit on the number of times other processes can enter their critical sections after a process has made a request to enter and before that request is granted, preventing indefinite starvation.

Q25: Explain the concept of Race Condition with a real-world programming example.

A **Race Condition** is an undesirable situation where the output of concurrent execution depends directly on the arbitrary timing or ordering of how threads are interleaved.

Example: Consider two threads executing a counter increment `counter++` simultaneously where the initial value of counter is 5.

```
// Assembly representation of counter++
1. REG = counter;
2. REG = REG + 1;
3. counter = REG;
```

If Thread A executes steps 1 and 2 (REG becomes 6), but gets context-switched out before step 3, Thread B might execute steps 1, 2, and 3 completely (reading 5, incrementing to 6, and saving 6 to memory). When Thread A resumes, it executes step 3 and writes its saved register value (6) back to memory. Despite two distinct increments, the counter reads 6 instead of 7.

Q26: What is a Mutex (Mutual Exclusion lock), and how does it differ from a Semaphore?

- **Mutex:** A locking mechanism used to synchronize access to a single resource. It is an object with ****ownership semantics****: only the exact thread that successfully locked the mutex is permitted to unlock it.
- **Semaphore:** A signaling mechanism based on an integer counter variable. It does not have ownership semantics; any thread or interrupt can signal (increment) a semaphore.

Q27: Explain the difference between a Binary Semaphore and a Counting Semaphore.

- **Binary Semaphore:** Its integer value can only alternate strictly between 0 and 1. It behaves similarly to a mutex but lacks ownership constraints and priority inversion protection mechanisms.

- **Counting Semaphore:** Its integer value can span across an unrestricted domain. The value typically represents the number of available instances of a shared pool of resources. `wait()` decrements the counter, and `signal()` increments it. Threads block when the counter hits zero.

Q28: What is a Spinlock? When is it preferable to use a spinlock over a traditional mutex?

A **Spinlock** is a lock where a thread continuously checks a condition variable in a tight, busy-waiting loop (wasting CPU cycles) until the lock becomes available.

When it is preferable: It is ideal in multi-core architectures when the lock is expected to be held for very brief durations. In these scenarios, the overhead of context switching a thread out and waking it up later via a mutex takes longer than spinning in place for a few CPU cycles. Spinlocks are frequently used inside operating system kernels.

Q29: Describe the classic Producer-Consumer Problem (Bounded-Buffer Problem) and how semaphores solve it.

The problem describes two classes of threads: **Producers** (which insert data items into a fixed-size buffer) and **Consumers** (which extract data items out of it concurrently).

Semaphore Solution: We use three semaphores to coordinate access:

- `mutex` (initialized to 1): Enforces mutual exclusion on buffer manipulation.
- `empty` (initialized to N): Tracks available empty slots in the buffer.
- `full` (initialized to 0): Tracks filled data slots available for consumption.

```
// Producer Logic
wait(empty);
wait(mutex);
// Add item to buffer
signal(mutex);
signal(full);

// Consumer Logic
wait(full);
wait(mutex);
// Remove item from buffer
signal(mutex);
signal(empty);
```

Q30: What is the Dining Philosophers Problem, and what does it illustrate about resource allocation?

Five philosophers sit around a circular table with five chopsticks. Each philosopher needs two chopsticks (left and right) to eat. It illustrates the challenges of allocating limited resources among competing processes without causing deadlocks or starvation. If every philosopher sits down and simultaneously picks up their left chopstick, no right chopsticks are available, causing a permanent **Deadlock**.

Q31: What is Priority Inversion, and how does Priority Inheritance mitigate it?

Priority Inversion is a scenario where a low-priority thread holds a shared lock required by a high-priority thread, preventing the high-priority thread from running. The situation worsens if a medium-priority thread arrives; since it doesn't need the lock, it preempts the low-priority thread, indirectly delaying the high-priority thread indefinitely.

Mitigation (Priority Inheritance): The low-priority thread temporarily inherits the high priority level of the waiting thread the moment the high-priority thread blocks on the lock. This prevents medium-priority tasks from preempting it, allowing it to finish execution and release the lock quickly.

4. Deadlocks

Q32: What is a Deadlock? What are the four Coffman conditions that must hold simultaneously for a deadlock to occur?

A **Deadlock** is an execution state where a set of processes are permanently blocked because each process holds a resource and waits for another resource held by someone else in that same set.

The **four Coffman conditions** are:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one process can use it at a time).
2. **Hold and Wait:** A process must be actively holding at least one resource while waiting to acquire additional resources held by others.
3. **No Preemption:** Resources cannot be forcibly taken from a process; they can only be released voluntarily.
4. **Circular Wait:** A closed chain of processes exists, where each process waits for a resource held by the next process in the chain.

Q33: How does Deadlock Prevention differ from Deadlock Avoidance?

- **Deadlock Prevention:** A design approach that structurally eliminates the possibility of deadlocks by invalidating at least one of the four Coffman conditions (e.g., forcing processes to request all resources upfront to eliminate "Hold and Wait"). This often results in low resource utilization.
- **Deadlock Avoidance:** A dynamic approach where the OS tracks resource allocation states in real time. For every resource request, the OS checks if granting it would put the system into an "Unsafe State" that could lead to a deadlock. It requires the system to have prior knowledge of each process's maximum resource needs (e.g., Banker's Algorithm).

Q34: Explain Banker's Algorithm. What is a "Safe State"?

Banker's Algorithm is a classic deadlock avoidance algorithm designed for systems with multiple instances of resources. It tests all incoming requests by simulating allocation. It calculates if an execution sequence exists where every process can satisfy its maximum resource requirements and terminate safely one by one.

A **Safe State** means the OS can guarantee that all processes can eventually finish execution without

deadlocking, even if they all suddenly request their maximum declared resource allocations. An unsafe state is not a deadlock, but it carries a risk of transitioning into one.

Q35: How does an OS handle Deadlock Detection and Recovery?

Instead of actively preventing or avoiding deadlocks, the OS lets them happen, periodically runs a detection routine, and recovers if needed:

- **Detection:** The OS maintains a directed **Resource Allocation Graph (RAG)**. It regularly runs cycle-detection algorithms (like Tarjan's or DFS) to locate circular wait dependencies.
- **Recovery Mechanisms:**
 - **Process Termination:** Aborting all deadlocked processes, or aborting them one by one until the cycle breaks.
 - **Resource Preemption:** Forcibly reclaiming a resource from a process and rolling its state back to a previous checkpoint to retry later.

Q36: What is the Ignorance / Ostrich Algorithm in the context of deadlocks?

The **Ostrich Algorithm** is a strategy where the OS completely ignores the deadlock problem, pretending deadlocks never happen. This is based on the trade-off that deadlocks are rare in standard environments, and the performance cost of constantly running prevention or avoidance logic outweighs the occasional manual reboot. Modern consumer systems like Linux and Windows use this strategy for general applications.

Q37: What is the difference between Deadlock and Livelock?

- **Deadlock:** Processes are stuck in a **blocked/waiting state** and do not consume CPU cycles. They are completely frozen waiting for resources that will never be released.
- **Livelock:** Processes actively change their internal states and consume CPU cycles, but make no actual forward progress. This often happens when two threads try to avoid a deadlock by continuously backing off and reacting to each other's state changes in lockstep (e.g., two people trying to pass each other in a hallway, repeatedly stepping to the same side at the same time).

5. Memory Management

Q38: Explain the difference between Logical (Virtual) Address and Physical Address.

- **Logical (Virtual) Address:** An address reference generated by the CPU from the perspective of an executing program. Programs see a unified, continuous block of memory starting at address 0, independent of actual hardware layout.
- **Physical Address:** The actual hardware address location on the physical RAM chips. The program cannot see or interact with these addresses directly.

Q39: What is the function of the MMU (Memory Management Unit)?

The **MMU** is a dedicated hardware component inside or close to the CPU. Its job is to translate virtual memory addresses into physical RAM addresses in real-time during instruction execution. It references system **Page Tables** to perform this mapping and checks access privileges, throwing a hardware trap if an invalid memory access is attempted.

Q40: Explain the concept of Paging. What is a Page Table, and where is it located?

Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory. It splits virtual memory into fixed-size chunks called **Pages**, and physical memory into identical blocks called **Frames**.

A **Page Table** is an in-memory data structure that maps a program's virtual pages to physical hardware frames. Each process has its own page table, which is stored in main physical memory (RAM). The CPU tracks its location using a dedicated register (e.g., CR3 in x86 architectures).

Q41: What is the TLB (Translation Lookaside Buffer), and how does it speed up memory access?

Because standard page tables are stored in RAM, every data access requires two lookups: one to read the page table entry, and another to fetch the actual data.

The **TLB** is a fast, hardware associative cache built straight into the CPU core. It caches recent virtual-to-physical address translations. When the CPU requests an address, it checks the TLB first. A **TLB Hit** skips the page table lookup, reducing translation times to near-zero.

Q42: Contrast Paging with Segmentation. Can they be combined?

- **Paging:** Memory is split into fixed-size, arbitrary blocks. It is transparent to the programmer and avoids external fragmentation, though it can cause minor internal fragmentation.
- **Segmentation:** Memory is split into variable-sized blocks based on logical programming divisions (e.g., Code, Heap, Stack, Library blocks). It is visible to the programmer but can lead to external fragmentation.

Combination: Yes, modern architectures use **Paged Segmentation** (e.g., x86-64). The logical space is divided into segments for logical protection, and each segment is further broken down into uniform pages for clean physical allocation.

Q43: What is Virtual Memory, and how does it allow programs larger than physical RAM to execute?

Virtual Memory is an abstraction technique that detaches an application's logical memory space from physical storage limitations. It uses disk storage (swap space/pagefiles) as an extension of physical RAM. Only the active, executing pages of a process need to be loaded into physical RAM at any given moment; inactive pages are swapped out to disk, allowing the system to run applications that exceed total physical memory capacity.

Q44: What is a Page Fault? Step through how the OS handles a page fault.

A **Page Fault** is a hardware interrupt raised by the MMU when a running program tries to access a virtual page that is valid but not currently loaded into physical RAM.

Handling Steps:

1. The MMU traps to the OS kernel.
2. The kernel shifts the process into a waiting state and checks its internal structures to verify if the address request is valid.
3. The kernel locates the missing page on secondary disk storage.
4. The kernel finds an empty physical frame in RAM. If RAM is full, it runs a page replacement algorithm to evict an existing page.
5. The kernel reads the page from disk into the physical RAM frame.
6. The process's page table is updated, marking its presence bit as valid.
7. The CPU restarts the exact instruction that triggered the fault.

Q45: Explain the following page replacement algorithms: FIFO, LRU, and Optimal Page Replacement. What is Belady's Anomaly?

When RAM is full, the OS must choose which page to evict:

- **FIFO (First-In, First-Out):** Evicts the oldest page loaded into memory. Simple to implement but can perform poorly.
- **LRU (Least Recently Used):** Evicts the page that hasn't been accessed for the longest duration. Good approximation of optimal behavior but requires hardware tracking support.
- **Optimal (OPT):** Evicts the page that will not be used for the longest time in the future. It is provably optimal but impossible to implement in practice because it requires future knowledge.

Belady's Anomaly: A counterintuitive phenomenon where increasing the number of physical memory frames results in *more* page faults. This occurs in FIFO page replacement, but cannot happen in stack-based algorithms like LRU.

Q46: What is Thrashing, and how can an OS prevent or fix it?

Thrashing occurs when an OS spends more time swapping pages in and out of disk storage than executing actual application instructions. This happens when the total working sets of all running processes exceed available physical RAM.

Prevention: The OS can track the **Working Set Model** or monitor page fault frequencies. If thrashing is detected, the medium-term scheduler swaps out entire processes to free up RAM frames for the remaining tasks.

6. Storage & File Systems

Q47: What is a File Allocation Table (FAT) vs. an Inode structure in Unix-like systems?

- **FAT (File Allocation Table):** A simple file system design where the entire disk's block layout is tracked in a single linear table. File entries point to their first block index, and each entry in the table points to the next block index, forming a linked chain.
- **Inode (Index Node):** A data structure used in Unix-like systems that describes a file object. It stores metadata (permissions, owner, size) and an array of block pointers. It features direct pointers for small files, and indirect/double-indirect pointers to map large files efficiently, allowing faster random access.

Q48: Explain the different file allocation methods: Contiguous, Linked, and Indexed Allocation.

- **Contiguous Allocation:** Every file occupies a linear, continuous set of blocks on disk.
Pros/Cons: Fast sequential and random read speeds, but suffers from heavy external fragmentation and requires knowing file sizes upfront.
- **Linked Allocation:** Dispersed disk blocks are linked together via pointers embedded within the blocks themselves.
Pros/Cons: No external fragmentation, but random access is slow because you must traverse the chain sequentially. A single corrupted pointer breaks the file.
- **Indexed Allocation:** Collects all block pointers into a single dedicated block called the ****Index Block****.
Pros/Cons: Supports fast random access without external fragmentation, but wastes space on index blocks for very small files.

Q49: What is Internal Fragmentation vs. External Fragmentation? How does Compaction address the latter?

- **Internal Fragmentation:** Unused space trapped **inside** a fixed-size allocated memory block (e.g., assigning a 4KB page to store a 1KB file leaves 3KB wasted).
- **External Fragmentation:** Dynamic allocation leaves plenty of total free memory space, but it is broken up into tiny, non-contiguous fragments across the disk or RAM. As a result, a new allocation request may fail even if there is enough total free space available.

Compaction: A shuffle mechanism that moves all allocated memory blocks to one side of the address space, consolidating all scattered free spaces into one large, continuous block. This requires dynamic address relocation capabilities.

Q50: What is RAID? Briefly explain the differences between RAID 0, RAID 1, and RAID 5.

RAID (Redundant Array of Independent Disks) combines multiple physical hard drives into a single logical unit to improve performance, reliability, or both.

- **RAID 0 (Striping):** Data blocks are split across multiple drives. It offers high read/write performance but no fault tolerance; if one drive fails, all data is lost.
- **RAID 1 (Mirroring):** Automatically duplicates data across two or more drives. Provides high fault tolerance and read speeds, but doubles storage costs.
- **RAID 5 (Block-Interleaved Distributed Parity):** Strips data blocks across at least three drives along with distributed parity blocks. It balances performance and fault tolerance, allowing the array to recover data if a single drive fails.